



Wavefront Alias .OBJ loader for Processing

by [Tatsuya SAITO](#) and [Polymonkey](#)

rev 13

Updates

- 07/01/2007 : rev13 added some material features (by polymonkey)
- 05/29/2007 : rev12 fixed a texture coordinate bug. texture swapping (thanks to polymonkey)
- 03/10/2007 : rev11 fixed some major bugs.
- 01/30/2006 : rev10 fixed some bugs.
- 11/16/2005 : rev09 supported Processing 95+.
- 06/18/2005 : rev07 added functions for model transformation
- 04/05/2005 : rev06 fixed a few bugs, supports alpha value
- 28/04/2005 : rev05 applet execution support
- 26/04/2005 : rev04 fixed a texture mapping problem
- 25/04/2005 : added FAQ
- 24/04/2005 : rev03 fixed a few bugs, 3ds exported .obj support
- 19/04/2005 : rev02 .mtl support, normal vertex support, lighting support
- 16/04/2005 : rev01 launched

Description

This is Wavefront Alias .OBJ file loader for Processing. It loads .OBJ model file and renders the model onto a screen.

Download

Download package from here ([objloader_r13.zip](#)).

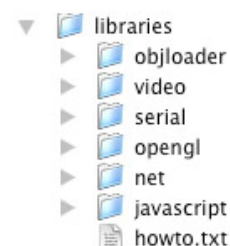
Sample project is available here ([ObjLoader_Sample.zip](#))

Source code is here ([source.zip](#))

Installation

After uncompressing the file, copy 'objloader/' folder into 'libraries/' folder which you can find under the Processing folder. Since the archive file is compressed in MacOSX environment, it might have some needless hidden files. Notice that the file you need is only 'objloader' folder and the files in the folder.

The folder/file structure should be as follows.



Restart Processing. You should be able to import the objloader library from sketch menu.

Reference

This library contains a class for loading/rendering a .OBJ file (OBJModel) and a class for accessing each vertex (Vertex) which can be primarily used to addressing vertexes in the model file and transform it.

the class has the following methods and member variables.

OBJModel

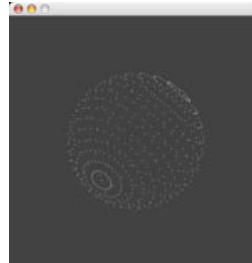
standard functions

`void load(String filename)` loads a model file.

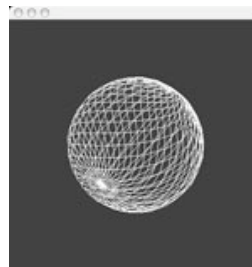
`void draw()` draws the model.

`void drawMode(int mode)` set render mode (ex. TRIANGLES, POLYGON, LINES etc.)
Basically the argument to this function is same as the one given to `beginShape()`. [Here](#) is a description in detail.

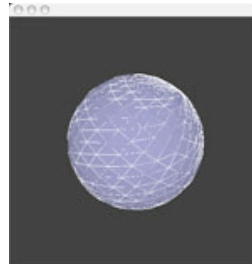
POINTS



LINES



TRIANGLES



`void setTexture(PImage)` swap the model texture with the specified PImage.

`void enableTexture()` enable texture rendering

`void disableTexture()` disable texture rendering

`int getVertexsize()` returns the number of vertexes that the model contains

`Vertex getVertex(int index)` returns the vertex data specified by array index

debug functions

`void debugMode()` set debug mode. once this function is called, the library outputs debug messages.

`void showModelInfo()` outputs model information, such as vertex data, facets structure and material info, in the debug window.

call `debugMode()` before using this function.

Vertex

float vx	x element of vertex
float vy	y element of vertex
float vz	z element of vertex

FAQ

What is .OBJ file?

.OBJ file is a standard 3D object file format by [Alias](#).

[Alias](#) is the leading software company of 3D graphics technology for the film, video, game development, interactive media, industrial design, automotive industry and visualization softwares. Their [.OBJ ASCII file format](#) is widely accepted all over the world as a standard format for exchanging data between 3D graphics applications. OBJ files contain solids which are made up of 3 or 4 sided faces and material data such as texture is searatedly stored in .MTL file.

What is .MTL file?

.OBJ file often has a link to .MTL file, which contains material information such as texture, color and surface reflection. To use the material information, you need to add .MTL file to the project. Currently, OBJModel class supports only texture information. Texture file images (.jpg) should be added to your project too.

Future updates will include support for other material information.

Polygons are not correctly rendered. What should I do?

The first thing you should make sure is what type of polygons your model is composed of. If it is made of quads, the correct option for drawMode() is POLYGON. If it is made of triangles, drawMode(TRIANGLES) still works. drawMode(TRIANGLES) is recommended for Processing BETA 85 because of some rendering bugs (shapes drawn as POLYGONS sometimes don't show up.) More detailed info is available here ([discourse: Libraries, Tools - New Libraries: OBJ Loader, Google Web API](#))

If the model still doesn't show up, please send me the Processing source code and .OBJ model files you are trying to render: tatsuyas@ucla.edu

Texture is not correctly shown. What should I do?

To render texture, three files have to be added to your project: .OBJ file, .MTL file, texture image file.

OBJModel uses standard Processing functions to render models. Inside the library, texture image is stored as PImage object, which means that the texture image file format has to be supported by PImage. Currently it supports .jpg.

I still have a problem with online execution

Hmm.. I updated the library so that it can load models files by relative path wherever a sketch program is located. It might still have a bug. Please send me the model file and the sketch program you have a problem with.

Examples

Simple rendering

```
// .OBJ Loader
// by SAITO <http://users.design.ucla.edu/~tatsuyas>
// Placing a virtual structure represented as mathematically
// three-dimensional object.
// OBJModel.load() reads structure data of the object stored
// as numerical data.
// OBJModel.draw() gives a visual form to the structure data.
// processing standard drawing functions can be used to manipulate
// the visual form to create deep visual experiences.
// Created 20 April 2005
```



[execute the example
applet](#)

```

import saito.objloader.*;
import processing.opengl.*;

OBJModel model;
float rotX;
float rotY;

void setup()
{
    size(400, 400, OPENGL);
    framerate(30);
    model = new OBJModel(this);
    model.load("dma.obj"); // dma.obj in data folder
}

void draw()
{
    background(51);
    noStroke();
    lights();

    pushMatrix();
    translate(width/2, height/2, 0);
    rotateX(rotY);
    rotateY(rotX);
    scale(20.0);
    model.drawMode(POLYGON);
    model.draw();
    popMatrix();
}

void keyPressed()
{
    if(key == 'a')
        model.enableTexture();
    else if(key=='b')
        model.disableTexture();
}

void mouseDragged()
{
    rotX += (mouseX - pmouseX) * 0.01;
    rotY -= (mouseY - pmouseY) * 0.01;
}

```

Model transformation

```

// .OBJ Loader transformation
// by SAITO <http://users.design.ucla.edu/~tatsuyas>
// Placing a virtual structure represented as mathematically
// three-dimensional object.
// OBJModel.getVertex() allows accessing to each vertex
// OBJModel.setVertex() allows transformation of a model

import saito.objloader.*;

OBJModel model;
OBJModel tmpmodel;

float rotX;
float rotY;

void setup()
{
    size(600, 600, P3D);
    framerate(30);
    model = new OBJModel(this);
    tmpmodel = new OBJModel(this);
    model.debugMode();
    model.load("dma.obj");
    tmpmodel.load("dma.obj");
}

void draw()
{
    background(255);
    lights();
    pushMatrix();
    translate(width/2, height/2, 0);

```



[execute the example
applet](#)

```
        rotateX(rotY);
        rotateY(rotX);
        scale(30.0);

        // renders the temporary model
        tmpmodel.draw();

        popMatrix();

        animation();
    }

    // transformation parameter
    float k = 0.0;

    // transforms the original model shape and stores transformed shape
    // into temporary model storage
    void animation(){

        for(int i = 0; i < model.getVertexsize(); i++){
            Vertex orgv = model.getVertex(i);
            Vertex tmpv = new Vertex();
            tmpv.vx = orgv.vx * (abs(sin(k+i*0.04)) * 0.3 + 1.0);
            tmpv.vy = orgv.vy * (abs(cos(k+i*0.04)) * 0.3 + 1.0);
            tmpv.vz = orgv.vz * (abs(cos(k/5.)) * 0.3 + 1.0);
            tmpmodel.setVertex(i, tmpv);
        }
        k+=0.1;
    }

    void mouseDragged()
    {
        rotX += (mouseX - pmouseX) * 0.01;
        rotY -= (mouseY - pmouseY) * 0.01;
    }
}
```

Etc.

Next version will support the following functions:

- rendering with a non-screen device (ex. AIExport)
- material data loading
- video texture

Bugs, opinions and complains? Reach me at tatsuyas@ucla.edu

SAITO at UCLA.edu